BookMate — 107 Interview Questions, Answered

How to use this: every answer is written the way you would *say* it — two to six sentences. The shaded **If pushed** box is extra detail, only for when the interviewer digs deeper. The numbers are the real ones from the code (`app/config.py`, `app/graph.py`, `app/retrieval/firewall.py`), so you can defend every one of them.

The one sentence everything hangs off: chunks are stored in reading order, so "everything I've read" is a contiguous prefix `E[:n_p]` of a NumPy matrix — the spoiler rule is enforced by which rows exist in the array being searched, not by a filter or by asking the model nicely.

Numbers to have ready: 320-word chunks, 40-word overlap, 384-dim `BAAI/bge-small-en-v1.5`, top-K 5, RRF k = 60, sufficiency floor 0.30, oracle threshold 0.62, 250 words per page, 9,000-character evidence budget, max 2 validation retries, temperature 0.0, 10 graph nodes, 81 test functions.

Tier 1 — Absolutely must know

1. Tell me about your BookMate project.

BookMate is a reading companion: you ask questions about a book you are in the middle of, and it answers without spoiling anything ahead of you. You set your reading position with a page slider, and it answers only from the part you have already read, with citations to the exact passages. Under the hood it is a RAG system — the book is chunked, embedded and stored in reading order, and retrieval only ever searches the slice of the embedding matrix that sits behind your position. The interesting part was making "don't spoil" a structural guarantee instead of a prompt instruction.

> **If pushed:** Stack: Python, FastAPI, SQLite + FTS5, NumPy, fastembed (bge-small, ONNX, runs locally), LangGraph for the agent flow, React + Vite front end. Deployed with Docker on Render (API) and Vercel (web), with CI on GitHub Actions.

2. What problem does BookMate solve?

When you are halfway through a novel and lose track — "who is this character again, why did she leave?" — you cannot Google it or ask ChatGPT, because both will happily tell you the ending. BookMate answers that question using only the chapters you have actually read, and when the answer is only later in the book, it says so instead of spoiling it.

3. Why did you build BookMate?

Two reasons. It is a real need I have had myself while reading. And technically it has a clean shape: "don't spoil" can be expressed exactly, as a position in the text, which let me turn a fuzzy product rule into a hard invariant and then write tests that prove it holds. Most RAG demos are "chat with a PDF"; I wanted one where retrieval itself had a correctness property to defend.

4. What is RAG?

Retrieval-Augmented Generation. Instead of relying on what the model memorised during training, you first retrieve relevant passages from your own data, put them into the prompt, and ask the model to answer from them. The model's job changes from "remember the fact" to "read this evidence and summarise it".

5. Why did you use RAG instead of directly asking an LLM?

Three reasons. The model may never have seen the book — a user upload is definitely not in its training data. Without evidence, models fill gaps with plausible guesses. And the big one for this project: RAG is the only design where I control what the model sees. If the ending is never put in the prompt, the model cannot quote it back. Asking an LLM directly gives it the whole book it memorised, spoilers included.

6. What makes your RAG system "temporal"?

Normal RAG retrieves from the whole corpus. Here every chunk has a position in the story, and every query carries an "as of" point — the reader's position. Retrieval runs over only the part of the corpus that existed up to that point, so the searchable window moves forward as the reader does. It is the same idea as an as-of query in a database: answer using only what was visible at that time.

7. How does BookMate prevent spoilers?

The unread part of the book is never in the data being searched. Chunks are stored in narrative order, so "already read" is the array slice `E[:n_p]`, and a similarity search over that slice physically cannot return row `n_p`. The keyword search path adds `c.idx < :n_p` inside the same SQL statement. Before anything reaches a prompt, `assert_safe()` re-checks every chunk and raises `FirewallBreach` if one is past the reader. And the answer must cite only passages it was given, or it is rejected.

> **If pushed:** `assert_safe` raises rather than silently filtering on purpose: a quiet filter would hide exactly the bug the system exists to prevent.

8. Explain the complete flow of a user query through BookMate.

The UI sends `POST /chat` with the session id, the question, and optionally a new position. The API saves the position, then runs the LangGraph pipeline: (1) classify the question's intent — a display badge only; (2) compute `n_p`, the number of chunks entirely behind the reader, and retrieve — vector search over `E[:n_p]` plus FTS5 keyword search with `idx < n_p`, fused with RRF, top 5, plus neighbouring chunks for context; (3) assess the evidence with a cheap cosine floor; (4) the answer model writes an answer with citations or replies `INSUFFICIENT`; (5) a validator checks every citation was actually supplied, retrying up to twice. If the evidence is insufficient, an air-gapped oracle checks whether the answer exists later and picks the right refusal. The final state is logged, and the API returns the answer, outcome, citations, and how many chunks were eligible vs withheld.

9. What are embeddings?

An embedding is a list of numbers — a vector — that represents the meaning of a piece of text. Texts with similar meaning produce vectors that point in similar directions. In BookMate each chunk becomes 384 numbers from `bge-small-en-v1.5`.

10. Why do you need embeddings in BookMate?

So I can search by meaning, not by exact words. A reader asks "why did she leave?" and the book says "she departed without a word" — no shared keywords, but the embeddings are close, so the right passage is found. Readers never phrase questions the way the author wrote the text.

11. How does similarity search work?

Embed every chunk once at ingest and store the vectors. At query time, embed the question with the same model, score it against every stored vector, sort by score, take the top K. In BookMate that is literally one line: `scores = E[:n_p] @ q`, then `argsort` to rank. A book is a few thousand chunks, so a brute-force scan takes a millisecond or two — no index needed.

12. What is cosine similarity?

The cosine of the angle between two vectors. It measures direction, not length, so a long passage and a short question can be compared fairly. It ranges from -1 to 1; for text embeddings you see roughly 0.3 to 0.9 in practice. I L2-normalise every vector when I store it, so cosine similarity becomes a plain dot product — which is why the whole search is one matrix multiply.

13. Why do you restrict retrieval to a prefix of the book?

Because reading is sequential. If you have read chunk 40, you have read chunks 0 through 39 too, so the allowed set is always an unbroken run from the start — a prefix. Storing chunks in reading order turns "what you may see" into a slice, `E[:n_p]`, and a slice cannot contain future rows. There is no filter to forget, because there is nothing to filter.

14. Why not retrieve the whole book and filter spoilers afterward?

Two reasons. Safety: with retrieve-then-filter, the spoiler text is sitting in a variable in my process, one log statement or one refactor away from a prompt. With a prefix it never exists there at all. Quality: if I take the top 5 over the whole book and four of them are from the future, filtering leaves me one passage. Searching the prefix gives five *usable* passages, because top 5 is computed inside the allowed set.

15. What happens when the answer isn't available in the pages the user has read?

Retrieval still returns the closest passages within the prefix, but the answer model reads them, sees they don't answer the question, and replies `INSUFFICIENT`. The graph then asks the spoiler oracle — which can see the whole book but returns only a true/false — whether a later part answers it. If yes, the reader gets "I can't answer that without giving away something that comes later — ask again once you're further in." If no, they get "I don't have enough to go on from the pages you've read yet." Either way, nothing is made up and nothing from the future is shown.

Tier 2 — Architecture

16. What are the main components of BookMate?

Six parts. **Ingestion** — find the book on Gutenberg or take an upload (txt/epub), clean it, split chapters, chunk in reading order. **Store** — SQLite for books, chunks, full-text index, sessions and logs; one NumPy `.npy` matrix per book for embeddings. **Retrieval** — the `SafeRetriever` firewall (vector + keyword, fused with RRF) and the oracle. **Agents** — the LLM client (Groq, Gemini, Ollama or an offline stub), the question analyser and the citation validator. **Orchestration** — the LangGraph workflow. **Interface** — the FastAPI server and the React client with the position slider.

17. Why did you use FastAPI?

It gives request validation for free through Pydantic models — the question length, the page bounds, the API-key length are all declared on the schema and bad input gets a 422 before my code runs. It generates interactive docs at `/docs`, it has `BackgroundTasks` which I use for ingest jobs, a lifespan hook for startup, and it is type-hint driven, which fits the rest of the codebase. It is also fast and the standard choice for Python ML backends.

18. Why did you use LangGraph?

Because the flow is genuinely a graph, not a chain. It has two conditional branches (answer vs oracle; spoiler refusal vs thin refusal) and a cycle (validate → answer again, bounded by a retry count). LangGraph expresses branches and loops directly, with a typed shared state, and each node is a plain function I can unit test. A linear chain cannot express the retry loop.

19. What are the nodes in your LangGraph workflow?

Ten nodes: `analyze_question`, `retrieve`, `assess_evidence`, `answer`, `validate`, `spoiler_oracle`, and four terminal nodes — `finalise`, `refuse_spoiler`, `refuse_thin` and `surface_error`. The main path is analyze → retrieve → assess → answer → validate → finalise.

20. What is the state being passed through the LangGraph workflow?

A `TypedDict` called `ReaderState`. Inputs: `session_id`, `book_id`, `question`, `position`. Analysis: `intent`. Retrieval: `evidence`, `n_p`, `n_withheld`, `evidence_sufficient`, `top_similarity`, `answered_later`. Generation: `draft`, `citations`, `retry_count`, `validation_reason`, `provider_error`. Output: `answer`, `outcome`, `prompt_text`. The position is written once at the entry and never recomputed, so every node agrees on where the reader is.

21. Why use a graph instead of a simple sequential Python function?

A plain function would work — it would be a while loop with if/else. The graph makes the control flow explicit and inspectable: I can see every branch, every node is testable in isolation, and the state is typed. The retry cycle has a clear bound, and adding a new branch (like `surface_error` for provider failures) is one node and one edge rather than threading a new condition through a nested function. For a flow with branches and a loop, that clarity is worth the dependency.

22. What is SQLite used for?

Everything except the vectors. Tables: `books` (metadata), `chunks` (text, chapter, `idx`, `word_start`, `word_end`), `chunks_fts` (an FTS5 full-text index for keyword search), `sessions` (each reader's book and position), `ingest_jobs` (background ingest status), and `conversations` — an audit log of every turn with the answer, outcome, evidence ids, and the exact prompt and its SHA-256 hash. It runs in WAL mode with a busy timeout, so readers are not blocked while an ingest is writing.

23. Where are the book contents stored?

The raw downloaded or uploaded file sits in `data/raw`. The processed text lives in SQLite's `chunks` table, one row per chunk with its word offsets, and is mirrored into the `chunks_fts` table for keyword search. For deployment, a prebuilt library of public-domain books ships inside the Docker image as a seed (`seed/companion.sqlite3` plus the vectors), because the free host has no persistent disk.

24. Where are the embeddings stored?

In one NumPy `.npy` file per book, under `data/db/vectors`. The file is an `(n_chunks × 384)` float32 matrix, L2-normalised, loaded into memory to search. I deliberately did not use a vector database — a whole book is a few thousand vectors, and keeping it as an ordered array is what makes the prefix slice possible.

25. How do you associate an embedding with its page/chunk?

By position. Row *i* of the matrix is the chunk with `idx = i`, and the chunk id encodes it too: `book_id::c00042`. That ordering is asserted at ingest — `idx` must increase monotonically with both `word_start` and `word_end` — rather than trusted. From the chunk row I get the chapter and word offsets, and the page is just `word_start // 250 + 1`.

26. How do you know which pages the user has already read?

The user tells us, with the slider. The position is stored per session as a word offset — the ground truth, because it is exact and edition-independent. Pages are a view on top: 250 words per page by default, or, if the reader declares their printed edition's page count, page N maps to the same fraction of the text, so the slider matches the book in their hand.

27. How does the system know which embeddings are eligible for retrieval?

One query: `SELECT COUNT(*) FROM chunks WHERE book_id = ? AND word_end <= ?`. That count is `n_p`, and the eligible embeddings are exactly the first `n_p` rows. I use `word_end`, not `word_start`: chunks overlap by 40 words, so a chunk that starts before the reader can still contain words they haven't reached. Requiring the whole chunk to be behind the reader is conservative by one chunk and keeps the eligible set an exact prefix.

28. What happens after retrieval?

The top 5 fused hits are expanded to include their immediate neighbours for context, de-duplicated, re-checked against the boundary, and sorted into narrative order — the order the reader met them, which makes far more sense to the model than relevance order. Then `assess_evidence` applies a cheap cosine floor, the passages are trimmed to fit the 9,000-character budget (weakest dropped first), and they go into the answer prompt, each labelled with its id.

29. How do you generate citations?

Each passage in the prompt is headed by its id, like `[chunk abc::c00042]`, and the system prompt requires every factual sentence to end with the short id, like `[c00042]`. After generation, the validator parses every bracketed id — tolerating formatting variants like `[c1, c2]` or full-width brackets — and checks each one is in the set of passages actually supplied. A foreign or missing citation fails validation and triggers a retry. The API returns the evidence with chapter, page, similarity and text, so the UI can show the source.

30. What happens if retrieval returns irrelevant information?

Three layers catch it. A cheap cosine floor (0.30) catches degenerate cases where nothing is remotely close. The answer model is told to reply `INSUFFICIENT` if the passages don't support an answer — it is the real judge, since it actually reads them. And the validator rejects any answer whose citations don't match supplied passages. If it still fails after two retries, the flow goes to the oracle and ends in a refusal, not a guess.

Tier 3 — RAG fundamentals

31. What is Retrieval-Augmented Generation?

A pattern where, before the model answers, you fetch relevant text from an external knowledge source and include it in the prompt. It gives the model fresh or private information it was never trained on, lets you cite sources, and — importantly for me — controls what the model can possibly say.

32. What are the two major stages of RAG?

Retrieval — find the passages relevant to the question. Generation — have the LLM write an answer using those passages. In practice there is also an offline indexing stage before both: chunk and embed the documents.

33. What is the retrieval stage?

Turning the question into a search and returning the top-K most relevant chunks. In BookMate: embed the question, cosine-score it against the eligible prefix, run an FTS5 keyword search over the same range, fuse the two rankings with Reciprocal Rank Fusion, and keep the top 5 plus neighbours.

34. What is the generation stage?

Building a prompt from the system instructions, the retrieved passages and the question, and having the LLM write the answer. In BookMate the answer is limited to four sentences, must cite passage ids, must paraphrase rather than quote, and is then validated before it is shown.

35. What is a document chunk?

A small, self-contained piece of a document — in my case about 320 words of consecutive text, cut on sentence boundaries and never across a chapter break. It is the unit that gets embedded, retrieved and cited.

36. Why do we chunk documents?

An embedding is a fixed-size vector, so embedding a whole book gives one blurry average of everything — useless for finding a specific scene. Models also have context limits, and you want only the relevant paragraphs in the prompt. In BookMate there is a third reason: chunks are the unit the reading boundary is defined on.

37. What happens if chunks are too large?

The embedding averages several topics, so it matches any specific question weakly and precision drops. Each retrieved chunk also carries a lot of irrelevant text, which wastes prompt budget — and on a rate-limited free tier that directly means fewer passages. I see this in my own eval: a one-sentence fact inside a big chunk gets diluted, which is why retrieval recall on my test novella is only about 50%.

38. What happens if chunks are too small?

Each chunk loses context — "she left" with no idea who "she" is — so a passage can match the query without containing the answer. You also get many more vectors to store and search. I started at 200 words and saw evidence cutting off mid-conversation, so I moved to 320 with neighbour expansion.

39. What is an embedding model?

A neural network — usually a small transformer encoder — trained so that texts with similar meaning map to nearby vectors. Mine is `BAAI/bge-small-en-v1.5`, run locally through fastembed on ONNX Runtime, so there is no API call or cost per embedding.

40. What does an embedding represent?

The meaning of the text as a point in a high-dimensional space — 384 dimensions for bge-small. Individual dimensions don't mean anything on their own; what matters is distance and direction: similar meanings are close together.

41. What is vector similarity?

A number saying how close two vectors are — typically cosine similarity, dot product, or Euclidean distance. In search it is the ranking signal: higher similarity means the chunk is more likely relevant to the question.

42. Cosine similarity vs Euclidean distance?

Cosine measures the angle (direction only); Euclidean measures straight-line distance, so it is affected by vector length. For text, direction carries the meaning, so cosine is the usual choice. And once vectors are normalised to length 1 — as mine are — the two give the same ranking, since `||a − b||² = 2 − 2·cos(a, b)`.

43. What is a vector database?

A database built to store embeddings and search them by similarity at scale — examples are pgvector, Qdrant, Milvus, Pinecone, Chroma. They use approximate-nearest-neighbour indexes such as HNSW so search stays fast over millions of vectors, and they support metadata filters, updates and persistence.

44. Why would you use a vector database?

When the collection is too large to scan by brute force, when many users or services share it, or when you need filtering, persistence and replication. I didn't need one: a book is a few thousand vectors and a NumPy scan takes a millisecond or two. More importantly, an ANN index can't be sliced like an array, so a vector DB would turn my structural prefix guarantee into a metadata filter. If I scale up I'd move to one, but I'd be explicit about that trade-off.

45. What is semantic search?

Search by meaning rather than exact words: embed the query and the documents, and return the documents whose embeddings are closest to the query's.

46. How is semantic search different from keyword search?

Keyword search (like BM25) matches literal words — fast, exact, great for names and rare terms, but blind to synonyms. Semantic search handles paraphrase but can blur an exact proper noun. I run both and fuse them with RRF: a question naming "Moriarty" gets the keyword hit, and a loosely worded question still gets the semantic one.

> **If pushed:** RRF fuses on rank alone, so the incomparable scores (cosine vs BM25) never need normalising.

Tier 4 — LLM questions

47. What is an LLM?

A Large Language Model — a transformer neural network trained on a huge amount of text to predict the next token. That simple objective, at scale, gives it the ability to answer questions, summarise, reason and follow instructions.

48. How does an LLM differ from a traditional ML model?

A traditional model is trained for one task on labelled data — say, a spam classifier — and outputs a label or a number. An LLM is pretrained once on general text and then steered to many tasks just by the prompt, without retraining. It generates open-ended text, and its output can vary, which is why you need validation around it.

49. What causes hallucinations in LLMs?

The model is trained to produce likely-sounding text, not verified truth. If it lacks the right information, it still produces a fluent answer by filling gaps with plausible patterns. Missing context, ambiguous questions, and higher temperature all make it worse.

50. Does RAG completely eliminate hallucinations?

No. It reduces them a lot, but the model can still misread a passage, combine two passages wrongly, or add facts from its own memory. That's why BookMate adds a validator on top of retrieval rather than trusting it.

51. How does RAG reduce hallucinations?

It gives the model the actual evidence, so it doesn't need to guess, and it lets you instruct the model to answer only from that evidence and cite it. Citations make answers checkable — and in BookMate they are checked automatically.

52. How would you make BookMate's answers more reliable?

What is already there: temperature 0, an "answer only from these passages or say INSUFFICIENT" system prompt, mandatory citations checked by a deterministic validator, an optional model-based entailment check, and a bounded retry. Next steps: a cross-encoder reranker so better passages reach the prompt, sentence-level checks that each sentence's citation actually supports it, and a larger labelled evaluation set so every change is measured rather than guessed.

53. What is a prompt?

The input text you give an LLM — instructions, context and the question. In RAG the prompt also contains the retrieved passages.

54. What is a system prompt?

The top-level instructions that set the model's role and rules for the whole conversation, separate from the user's message. BookMate's says: use only the given passages, cite the short id on every factual sentence, reply exactly `INSUFFICIENT` if they don't support an answer, never hint at what happens later, paraphrase rather than quote, name people explicitly, and use at most four sentences.

55. What is temperature in an LLM?

A setting that controls randomness. The model's scores for the next token are divided by the temperature before being turned into probabilities: low temperature sharpens the distribution so the most likely token almost always wins; high temperature flattens it so less likely tokens get picked more often.

56. What happens if temperature is increased?

Output becomes more varied and creative but less predictable, and more likely to drift from the evidence or invent details.

57. Why would you want a lower temperature for BookMate?

Because I want faithful, repeatable answers grounded in the passages, not creativity. BookMate uses temperature 0.0 for both the answer and the classifier. It also makes testing and debugging easier, since the same input gives nearly the same output.

Tier 5 — Your spoiler-prevention design

58. Explain your spoiler-prevention mechanism technically.

Four layers. (1) Ordered storage: chunks are embedded in narrative order, so row *i* of the matrix is chunk *i*. (2) The boundary: `n_p = COUNT(chunks WHERE word_end <= position)`. (3) Bounded search: the vector path searches `vectors.prefix(book, n_p)`, which is `E[:n_p]` — a structural bound; the keyword path has `c.idx < :n_p` in the same SQL statement as the FTS5 `MATCH` — a predicate bound. (4) A final `assert_safe()` before the prompt that raises `FirewallBreach` on any chunk past the reader. The only code that can see the future is the oracle, which returns a single boolean and is not an LLM call.

59. What exactly do you mean by "prefix slice"?

The first `n_p` rows of the embedding matrix — `E[:n_p]` in NumPy. It is a "prefix" because reading is sequential, so the allowed set always starts at the beginning and is unbroken. Example: a book of 500 chunks, reader 43 chunks in — the searchable matrix has shape `(43, 384)`. Chunks 44 to 499 aren't removed; they were never in the array.

60. Why is your approach safer than post-retrieval filtering?

A filter is a line of code that has to be correct at every call site, forever. A slice is an invariant enforced by the shape of the data: the unsafe state can't be represented. With a filter there is a moment where a spoiler chunk is a live variable in memory; with a slice there isn't. The one place I couldn't make it structural — the SQL keyword path — is exactly where I put a mutation test.

> **If pushed:** The mutation test deletes the `c.idx < :n_p` line and asserts the test suite goes red. A firewall test that still passes when you break the firewall proves nothing.

61. What happens if the user has read pages 1–50?

Page 50 ends at word 12,500 (250 words per page), so `position = 12,500`. `n_p` is the number of chunks whose last word is at or before 12,500 — roughly the first 44. Retrieval runs only over those rows. The chunk that straddles pages 50 and 51 is excluded, because part of it is unread.

62. Can the model access page 51?

No. Page 51 is not in the searched matrix, not returned by the keyword query, and `assert_safe()` would raise if it ever reached the prompt. The model only sees the prompt, and page 51 is never in it. The one caveat is the model's own training memory for famous books — covered in question 66.

63. What if page 51 has the most relevant answer?

Then the reader doesn't get it — that is the product working correctly. The answer model sees that pages 1–50 don't answer the question and replies `INSUFFICIENT`. The oracle then compares the question against the full book and finds the future material scores much higher (above 0.62 and at least 0.05 better than anything already read), so the reader is told "this is answered later — ask again once you're further in."

64. What should BookMate do when the answer exists only in unread pages?

Refuse honestly, without hinting at the content: "I can't answer that from what you've read so far without giving away something that comes later." It must not guess, foreshadow, or summarise the future — the system prompt forbids foreshadowing and the model never sees the future text anyway.

> **If pushed:** Honest limitation: saying "it's answered later" itself leaks about one bit ("yes, this gets resolved"). I accepted that because it's more useful than a flat "I don't know"; the stricter option is to show one identical refusal for

both cases.

65. How do you handle a question whose answer requires information from both read and unread pages?

BookMate answers only the part the read pages support. It retrieves only from the prefix, so the model can give a partial answer — "so far, X has happened [c00031]" — but it cannot mention anything from the unread part because it never sees it, and every sentence needs a citation to a supplied passage. If the read part isn't enough to say anything useful, it replies `INSUFFICIENT` and goes to the refusal path.

66. Could the LLM itself reveal a spoiler even if retrieval is restricted?

Yes — from its own training memory. If the book is *Frankenstein*, the model already knows the ending, and no retrieval firewall can un-train it. My guarantee is precise: the model is never *fed* future text, not that it never *knows* it. I reduce the risk with the system prompt ("if you think you recognise this book, that memory is off-limits"), temperature 0, and mandatory citations to supplied passages. And I measure what's left with a "parametric leakage" metric: ask the questions with zero evidence and count how many the model still gets right.

67. How would you test whether your spoiler boundary actually works?

I did test it, in four ways. (1) Unit tests: the vector operand is literally a prefix, no retrieved chunk is past the position, position 0 returns nothing, advancing the position only widens the eligible set. (2) A shingle leak sweep: take word sequences from future chunks and assert none of them ever appear in retrieved text, at every position across the book. (3) The mutation test on the SQL predicate. (4) An end-to-end test on a synthetic novella with facts planted at known positions, each tagged with a unique nonce word that appears exactly once in the book — so I can assert the nonce never reaches the prompt before its position, and does after.

> **If pushed:** Why synthetic: on a real novel a correct answer might come from the model's memory, so you can't tell a leak from recall. My first eval reported a 17% "leak" that was entirely a ground-truth mistake — the fact was keyed on a first name that appeared 225 times. Hence unique nonces.

Tier 6 — LangGraph

68. What is LangGraph?

A library from the LangChain team for building LLM workflows as a graph of steps. You define nodes (functions), edges (what runs next, possibly conditional), and a shared state that flows between them. It supports branches and cycles, which plain chains don't.

69. Why did you choose LangGraph for BookMate?

The workflow has real structure: two conditional branches and one bounded retry cycle. LangGraph expresses that directly, with a typed state, and each node stays a small, pure-ish function I can test on its own.

70. What is a node in LangGraph?

A step in the workflow — a function that receives the current state and returns a dict of updates to merge into it. For example, my `retrieve` node takes the question and position and returns `evidence`, `n_p` and `n_withheld`.

71. What is an edge?

A connection that decides which node runs next. A normal edge always goes to the same node (retrieve → assess_evidence). A conditional edge calls a routing function on the state and picks the next node — for example `route_after_validate` chooses between finalise, retry, oracle or error.

72. What is state in LangGraph?

The shared data object that flows through the graph. Every node reads from it and writes updates to it. Mine is a `TypedDict` called `ReaderState` holding the question, position, evidence, draft answer, retry count, outcome and so on.

73. What happens if the retrieval node finds no useful evidence?

`assess_evidence` marks it insufficient — either no passages at all, or the best cosine is under 0.30 — and the conditional edge skips the answer model entirely and goes straight to the spoiler oracle, which picks the "answered later" or "not enough yet" refusal. That also saves an LLM call.

74. How does your workflow decide whether to answer or refuse?

Two decision points. After retrieval, a cheap cosine floor decides whether it is even worth calling the answer model. After the answer, the model's own judgement decides: if it replied `INSUFFICIENT`, or validation still fails after two retries, the flow goes to the oracle and refuses. If it gave a grounded, correctly cited answer, it is finalised. A provider error goes to a separate error node — never dressed up as a refusal.

> **If pushed:** Why the model judges and not a threshold: I tried thresholds first. RRF scores encode rank only, so the gate passed everything; raw cosine was corpus-dependent — "what is the capital of Peru?" scores 0.455 against Sherlock Holmes. Only something that reads the passages can judge them.

75. Why not use LangChain alone?

LangChain's chains are mostly linear; my flow needs a retry loop and branches. Also, I deliberately didn't use LangChain's generic retrievers or vector stores: a generic retriever with a metadata filter is exactly the retrieve-then-filter pattern I was avoiding. The firewall is my own code because it is the part that has to be provably correct.

76. What advantage does a stateful workflow provide?

Every node sees the full context of the turn. The retry works because the validator writes `validation_reason` into the state and the answer node reads it back into the next prompt ("your previous draft was rejected because…"). `retry_count` bounds the loop. The position is set once, so every node agrees on the boundary. And at the end the whole state — including the exact prompt — is logged to the `conversations` table, which makes every answer auditable.

77. Could you implement BookMate without LangGraph? How?

Yes, easily — a single function: classify, retrieve, check evidence, then a `for attempt in range(3)` loop that calls the model and validates, with if/else branches to the oracle and the refusals. It would be maybe 50 lines. LangGraph doesn't give me anything I couldn't write; it gives clearer structure, easier testing of each step, and a flow I can draw.

Tier 7 — FastAPI/backend

78. What is FastAPI?

A modern Python web framework for building APIs. It uses type hints and Pydantic models to validate requests and serialise responses automatically, generates OpenAPI docs, and supports both sync and async endpoints.

79. Why FastAPI instead of Flask?

Built-in validation with Pydantic (Flask needs extra libraries), automatic interactive docs, native async support, dependency injection, and better performance on ASGI servers like Uvicorn. For a JSON API with strict input shapes, it is less code and fewer bugs.

80. What is a REST API?

An API style where resources are identified by URLs and manipulated with standard HTTP methods — GET to read, POST to create, PATCH to update, DELETE to remove — usually exchanging JSON. It is stateless: each request carries everything the server needs.

81. What endpoints would BookMate need?

It has them: `GET /health`; `GET /books`, `GET /books/{id}`, `DELETE /books/{id}`, `PATCH /books/{id}/edition` (declare your edition's page count); `POST /books/resolve` (search Gutenberg), `POST /books/ingest` and `POST /books/upload` (start a background ingest, returns 202 with a job id), `GET /books/ingest/{job_id}` (poll status); `POST /sessions`, `GET /sessions/{id}`, `PATCH /sessions/{id}/position`, `GET /sessions/{id}/history`; `POST /chat`; and `GET/POST/DELETE /config/api-key` for the LLM provider key.

82. What happens when the frontend sends a question to the backend?

The React app sends `POST /chat` with `{session_id, question, position}`. FastAPI validates it (question 2–500 characters), loads the session, and if the slider moved, saves the new position first so the session stays the single source of truth. It calls `ask()`, which runs the LangGraph pipeline. The response is a `ChatOut`: the answer, the outcome (answered / refused / error), the intent, how many chunks were eligible and withheld, and the evidence list with chapter, page, similarity and text for the citations.

83. How would you structure the API request?

As JSON matching a Pydantic model: `{"session_id": "…", "question": "Who is Irene Adler?", "position": 12500}`. The book comes from the session rather than the request, so a client can't ask about one book with another book's position. `position` is optional — if it is left out, the stored session position is used.

84. How would you validate user input?

Pydantic does the first layer: question length 2–500, edition pages 1 to 20,000, API key 8–512 characters and hidden from logs (`repr=False`), position must be non-negative. Uploads are capped at 25 MB. User text never reaches the FTS5 search raw — I extract words and quote each one, so search syntax can't be injected (there's a test for it). SQL always uses bound parameters. And missing sessions or books return clean 404s.

85. What is JWT authentication?

JWT is JSON Web Token — a signed token with three parts: header, payload (claims like user id and expiry) and signature. After login the server issues a token; the client sends it in the `Authorization: Bearer` header, and the server verifies the signature and expiry on each request without a session lookup. It is stateless, which suits APIs that scale horizontally.

> **If pushed:** Be honest: BookMate itself doesn't have user login yet — it is a single-user demo. I've used JWT elsewhere, and question 86 is how I'd add it here.

86. How would you secure the BookMate API?

Today it has a CORS allow-list, keys held only on the server (the browser sees just the last four characters), a `PERSIST_API_KEY=0` switch to keep pasted keys in memory only, and runtime ingest disabled on the free host. For real users I'd add: JWT login; tie each session to its user so nobody can read another person's history; restrict admin actions — deleting books, ingesting, setting the provider key — to an admin role, since those are currently open; per-user rate limits; HTTPS only; and secrets in a secret manager.

87. How would you handle multiple users simultaneously?

Each user has their own session row with their own position, and each request builds its own graph state, so there is no shared mutable state between users. FastAPI runs sync endpoints in a thread pool. SQLite is in WAL mode, so reads don't block during a write. The embedding model is guarded by a lock because ONNX sessions aren't thread-safe. And a per-model token bucket tracks the provider's tokens-per-minute limit, so concurrent questions queue briefly instead of failing with 429 errors.

Tier 8 — Failure cases / engineering questions

88. What happens if the retrieved chunks are irrelevant?

The answer model reads them and replies `INSUFFICIENT` instead of forcing an answer; the oracle then picks the right refusal. If the model tries to answer anyway, its citations must point to supplied passages, and the optional entailment check can reject unsupported claims.

89. What happens if there is no relevant information?

The same path, ending in the "not enough to go on" refusal — the oracle finds nothing in the future either, so it is `refused_thin`, not "answered later". An honest "I don't know" is the correct output.

90. What happens if the LLM hallucinates?

If it cites an id that wasn't supplied, or cites nothing, the deterministic validator rejects the draft and the answer node retries with the reason added to the prompt — up to two retries, then a refusal. An empty draft is also rejected. What it can't fully catch is a wrong claim wrongly attached to a real citation; the optional model-based entailment check is the second line for that.

> **If pushed:** A bug I fixed: an empty answer once passed validation, because it made no unsupported claims. Vacuous truth isn't grounding.

91. What happens if the embedding model fails?

At ingest, an `EmbedError` fails the job, which is reported as `failed` — nothing half-written is left looking like a valid book. At query time the question can't be embedded, so the request fails with an error rather than answering. A possible improvement is falling back to keyword-only search, which is still spoiler-safe because the keyword path has its own bound.

92. What happens if the database is unavailable?

SQLite is a local file, so "unavailable" usually means locked by a long write. Connections wait up to 30 seconds (busy timeout) instead of failing instantly, and WAL mode lets reads continue during writes. If the data directory is empty — as on a free host with no disk — the seed library baked into the Docker image is used, so the API still comes up with books.

93. What happens if the user asks 100 questions simultaneously?

Honestly, it would queue and slow down, and some requests would time out. The bottleneck is the LLM provider's free tier: about 8,000 tokens per minute, and one question costs about 2,600 tokens, so roughly three answers a minute per model. My token bucket makes requests wait for budget instead of all failing with 429s, and rate-limit retries follow the provider's `retry-after` header. To really handle it: a paid tier or several providers, a request queue, per-user rate limits, and caching repeated answers.

94. How would you reduce latency?

Almost all latency is the LLM call; embedding the question and searching take milliseconds. Already done: the intent classifier uses a keyword table instead of an LLM call by default, and the entailment check can be turned off. Next: stream the answer token by token, use a faster model for the answer, and cache answers for repeated questions.

95. How would you reduce LLM API costs?

Tokens are the cost. I already cap evidence at 9,000 characters (about 90% of the prompt), skip the LLM for intent classification, cap completion tokens per role, and route cheap tasks to a smaller model. Next: cache answers — but the cache key must include `n_p`, otherwise a reader further ahead could fill the cache with something a reader behind them shouldn't see.

96. How would you scale BookMate to 100,000 users?

Make the API stateless and run several instances behind a load balancer. Move from SQLite to Postgres, and embeddings to pgvector or a vector DB with the book id as the partition. Keep per-book brute-force search where possible, since it keeps the prefix guarantee; otherwise use an `idx < n_p` filter and make the tests around it load-bearing. Move ingest to a job queue with separate workers. Use a paid or multi-provider LLM setup, add an answer cache, and add monitoring. Users are the easy part — a position is just a row per user.

97. What would you change before deploying it to production?

Authentication and authorization, especially locking down the admin endpoints. A real database with backups instead of a file on an ephemeral disk. Rate limiting per user. Monitoring, logging and alerts on the error outcome. Removing slow external calls (the Gutenberg search) from the request path. A reranker for answer quality. And running the spoiler eval in CI on every change, so the firewall is checked before each deploy.

Tier 9 — "I want to see if you actually built it"

98. Show me the architecture of BookMate.

Draw this, and talk through it top to bottom:

```
React (Vercel)  --POST /chat {session_id, question, position}-->  FastAPI (Render)
                                                                        |
                                                                  graph.ask()
                                                                        |
  analyze_question -> retrieve -> assess_evidence
                          |              |   sufficient          |  thin
                          |              v                       v
                          |           answer <----+       spoiler_oracle (Store B:
                          |              |        | retry       full book, returns
                          |              v        | (<= 2)      ONE boolean)
                          |           validate ---+            /          \
                          |              |                refuse_spoiler  refuse_thin
                          |           finalise   (provider error -> surface_error)
                          |
        SafeRetriever (Store A: only chunks with word_end <= position)
          vector : E[:n_p] @ q        <- structural bound (NumPy slice)
          keyword: FTS5 ... idx < n_p <- predicate bound (SQL)
          RRF fuse -> top 5 -> neighbours -> narrative order -> assert_safe()

Storage: SQLite (books, chunks, chunks_fts, sessions, ingest_jobs,
         conversations) + one .npy matrix per book, row i = chunk i
```

99. Walk me through your code.

Follow one request. `app/api.py` — the FastAPI routes; `/chat` saves the position and calls `ask()`. `app/graph.py` — the LangGraph state, the ten nodes, and the routing functions. `app/retrieval/firewall.py` — the `SafeRetriever`, the only way to read book text; it can't be built without a position. `app/retrieval/hybrid.py` — RRF. `app/retrieval/oracle.py` — the air-gapped future check. `app/agents/` — `llm.py` (providers, retries, token budget), `prompts.py`, `validator.py` (citation check). `app/store/` — `db.py` (SQLite schema), `vectors.py` (the prefix slice), `embedders.py` (fastembed). `app/ingestion/` — sources, parsers, cleaner, chunker, pipeline. `tests/` — firewall, graph, spoilers and more. `scripts/` — ingest, the synthetic novella, the evaluator, the seed builder.

100. Which part of the project did you personally implement?

I designed and built the whole thing — it is a solo project. The parts I'd point to as most my own thinking: the prefix-slice firewall and the `word_end` boundary rule, the air-gapped oracle, the citation validator, the LangGraph flow with its separate error outcome, the synthetic-novella evaluation with nonces, and the deployment work to fit it into a 512 MB free instance.

> **If pushed:** Say only what is true. If you used AI tools for parts of the code, it's fine to say so — then explain how you checked and understood it. Don't claim a line you can't explain.

101. What was the hardest technical problem you encountered?

Deciding when to refuse. My first design thresholded the RRF score to judge whether the evidence was good enough — but RRF only encodes rank, so the top hit scores the same whether it's relevant or not, and the gate passed everything; the spoiler branch was unreachable. Raw cosine was better but corpus-dependent: "what is the capital of Peru?" scored 0.455 against Sherlock Holmes, close to real questions. A z-score didn't separate them either. The fix was to move the judgement to the thing that can actually read the passages — the answer model says `INSUFFICIENT` — and keep the cosine floor only as a cheap screen.

102. What bug took you the longest to solve?

The server getting killed for running out of memory on every upload. The free host caps memory at 512 MB. It took measuring to find three causes: setting `parallel=1` in fastembed actually spawned a second process with a second copy of the model (~800 MB total); a batch size of 32 made ONNX's memory arena grow and never shrink; and `os.cpu_count()` reported the host's 16 cores inside the container, so it sized per-thread buffers for 16. Fixes: embed in-process, batch 4–8, one or two threads. Peak memory went from ~800 MB to ~300 MB.

> **If pushed:** Another good one: provider errors were being caught and returned as `INSUFFICIENT`, so an outage scored as 100% refusal accuracy. A broken system looked perfectly safe. That's why the separate `error` outcome exists.

103. What design decision would you change now?

I'd build the evaluation set before the features. Twice I fixed the wrong thing because I had no measurement — the score-threshold gate and the 17% fake leak. I'd also change the chunking: 320-word chunks dilute one-sentence facts, and recall on my novella is about 50%. I'd embed smaller pieces (a few sentences) for matching but return the surrounding larger chunk as evidence. That keeps the prefix property, since small pieces are also in reading order.

104. What did you learn from building this?

Put the safety property in the data structure, not in a prompt or a filter — and then try to break it with a mutation test. Make sure your success metric can actually fail: the empty answer that "validated" and the outage that scored as perfect refusals both taught me that. Measure before and after every change — the memory work only succeeded because every change had a number. And real deployment constraints — memory, CPU, rate limits — shape the design as much as the algorithm does.

105. What is the biggest limitation of your current implementation?

The model's own memory of famous books. The firewall guarantees the model is never fed future text, but it can't make it forget *Frankenstein*; I reduce and measure that, not eliminate it. Close second: retrieval recall — about 50% on the test novella — because facts get diluted in large chunks. And on the engineering side: no user authentication and a single SQLite file, so it's a demo, not a multi-user product yet.

106. If I gave you one week to improve BookMate, what would you do?

Days 1–2: add a cross-encoder reranker over a larger candidate pool (still inside the prefix) and try smaller matching chunks, measuring recall on the novella before and after. Day 3: run the spoiler eval in CI so every change is checked. Day 4: stream answers to cut perceived latency. Day 5: add JWT login, bind sessions to users, and lock down the admin endpoints.

107. If you had to rebuild it from scratch, what would you change?

Start with the evaluation harness and the synthetic book on day one. Use Postgres from the start instead of SQLite, with per-book brute-force vector search to keep the prefix guarantee. Run ingest on a separate worker from the beginning, since embedding a book is minutes of CPU and shouldn't share a box with the API. Design for authentication from day one. The core idea — ordered storage and a prefix slice — I would keep exactly as it is.